National University of Computer and Emerging Sciences

Information Retrieval (CS4051)

Date: February 23, 2026

Course Instructor

Muhammad Rafi

Sessional-I Exam

Total Time: 1 Hour

Total Marks: 40

Total Questions: **03**

Semester: Spring-2026

Campus: Karachi

Department: AI & DS

_____ _____ _____ _____

Student Name Roll No Section Student Signature

_____ _____

Vetted by Vetter Signature

CLO # 1: < Understand the basic concepts and techniques in Information Retrieval.>
CLO # 4 < Appreciate the importance of data structures, indexes and retrieval efficiency>

Q1. Answer the following questions to the point. **[Time: 30 mins] [Marks: 2x10]**

a. What is an Inverted Index? What are its two components?

An inverted index a data structure most suitable for information retrieval task, it has two components (i) Dictionary or vocabulary or lexicon and (ii) Posting lists.

b. Outline two differences between Dictionary and Thesaurus – give one use of each.

Dictionary	Thesaurus
- The main purpose of the dictionary is to support understanding of a word, it gives meaning and definition of the word. - Usage: Need to know the meaning of a given word	- The main purpose of Thesaurus is to give a clear idea of usage of a word in a language. It mostly contains usage, related words (synonyms/antonyms/etc) It is used to support language understanding - Usage: understand word and related words

c. Why precision and recall are not good for evaluating an IR task?

Both precision and recall do not consider position of the document in the result-set. Hence two same level of precision and recall may have different user experience from the result-set. On the other side, implementer of the IR systems can easily manipulate the two values by set the retrieval and relevant set.

d. Explain the syntax and semantic of proximity query in IR.

The most common syntax for the proximity query is "t1 t2 /k" where the semantic is to Retrieve documents where term1 and term2 occur within k positions (word distance $\leq$ k).

e. Explain Heaps' law. How it helps in estimating an inverted index?

Heaps' law: vocabulary size grows sublinearly with collection size:
$V(N)=kN^{\beta}$, $0<\beta<1$ where N = number of tokens, V=unique terms.
Heaps law helps in estimate dictionary size (and memory/storage needs) and indirectly plan index size and scaling as documents grow.

f. What are the two motivation to compressed a dictionary of an inverted index?

Two main motivation to compressed a dictionary part of an inverted index is to (i) space memory/cache – larger size dictionary can now fit in RAM and thus enable faster query processing. (ii) It also reduce disk I/O and storage cost.

g. Define Jaccard index or similarity also gives it formula.

Jaccard index or similarity is a function in set-algebra to compute similarity between two sets say A and B. Jaccard (A,B)= $|A \cap B| / |A \cup B|$ Range: 0 (no overlap) to 1 (identical). Used for shingles, near-duplicate detection in IR.

h. What is Truecasing? where it is used?

Truecasing is the idea to restores the most likely capitalization of words in text (e.g., "pakistan" → "Pakistan"). This is mainly used in NLP pipelines such as machine translation, ASR transcripts, NER, and preprocessing when case information is helpful in solving the task e.g. Named Entity.

i. Why are skip pointers not useful for queries of the form x or y?

The use of skip pointers in posting part of an invested index is very helpful in quickly getting the result-set of a query of type t1 AND t2, as it need to be a common document that contains both the terms. So, the intersect part quickly move to the higher index to get a match. This is not very beneficial for the query of the type t1 OR t2.

j. Suggest some use case where search engines are effectively combine both bi-word index and positional index together?

In modern information retrieval systems – 12% - 15 % queries are phrase queries and mostly about the named entities. Both bi-word index and positional index can be used

wisely to scale the performance. The most frequent phrases like celebrity's names "Will Smiths" can be access via bi-word index and the less frequent phrases can be accessed by positional index. In this way a lot of saving can also be done.

CLO # 2: <Understanding how IR Model works >

Q2. Consider the following documents in a collection. **[Time: 15 mins] [Marks: 2x5]**

d1: smart cities deploy sensors for traffic and air quality
d2: cybersecurity teams monitor network traffic
d3: air quality improves when cities reduce traffic emissions.

a. Prepare a vocabulary from the above corpus. [1]

V = [air, and, cities, cybersecurity, deploy, emissions, for, improves, monitor, network, quality, reduce, sensors, smart, teams, traffic, when]
|V|=17

b. Prepare a term document matrix for the above collection. [1]

Term/Doc	d1	d2	d3
air	1	0	1
and	1	0	0
cities	1	0	1
cybersecurity	0	1	0
deploy	1	0	0
emissions	0	0	1
for	1	0	0
improves	0	0	1
monitor	0	1	0
network	0	1	0
quality	1	0	1
reduce	0	0	1
sensors	1	0	0
smart	1	0	0
teams	0	1	0
traffic	1	1	1
when	0	0	1

c. Prepare a posting list for the above corpus. [2]

air → {d1, d3}
and → {d1}
cities → {d1, d3}
cybersecurity → {d2}
deploy → {d1}
emissions → {d3}
for → {d1}
improves → {d3}

monitor → {d2}
network → {d2}
quality → {d1, d3}
reduce → {d3}
sensors → {d1}
smart → {d1}
teams → {d2}
traffic → {d1, d2, d3}
when → {d3}

d. Prepare document vectors using vocabulary from **part (a)** and using term frequency only, for documents d1 and d3. [2]

d1=[1,1,1,0,1,0,1,0,0,0,1,0,1,1,0,1,0]
d3=[1,0,1,0,0,1,0,1,0,0,1,1,0,0,0,1,1]

e. Prepare query vector for the query **q** = "traffic" **AND** "air", using the term frequency only? [2]

q=[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0]

f. Find the cos(angle) between document **d1** and query **q** in part (e). [2]

cos(d1,q) = { d1 . q } / { |d1| x|q| } where d1 . q = 2 ; |d1| = 3 and |q| = √2
Hence **cos(d1,q) = √2 / 3 = 0.471**

CLO # 3: < Appreciate the importance of data structures, indexes and retrieval efficiency >

Q3. Answer the following questions. **[Time: 15 mins] [Marks: 2x5]**

a. Construct a bi-gram (2-gram) index (use boundary symbol $) for the terms - {secure, sensor, semantic}

Bi-gram	**Words**
$s	{secure, sensor, semantic}
an	{semantic}
c$	{semantic}
cu	{secure}
e$	{secure}
ec	{secure}
em	{semantic}
en	{sensor}
ic	{semantic}
ma	{semantic}
ns	{sensor}
nt	{semantic}
or	{sensor}
r$	{sensor}
re	{secure}
se	{secure, sensor, semantic}
so	{sensor}
ti	{semantic}
ur	{secure}

b. What is a permuterm index? explain what permuterm index you would search if the wildcard query is of type dol*lar is processed through it?

Permuterm index is an index over all rotations of each term appended with a boundary symbol $. It converts a wildcard query with a single * into a prefix query.
For query dol*lar, the pattern will prefix = "dol", suffix = "lar" and Convert to permuterm form by rotating around * to get the suffix$prefix="lar$dol"

c. Why sometime a post processing is necessary in information retrieval? How can we reduce the cost of post processing?

The post processing in IR is a specialized phase of result-set processing. In some model of IR there is a chance of getting false positive documents against a given query due to the processing approach used is very general. The post processing tries to filter all the false positive documents.

The complexity of this phase is directly proportional to number of documents in the result-set, we can reduce the amount of work required to do post processing by keeping the results-set small.

d. Explain the idea of context sensitive spelling correction. Give one example of the query where context sensitive spelling correction is required.

The key idea behind context sensitive spelling correction is to use surrounding query context and making the correction of individual words based on the whole query most probable value for an example: consider the query "flew form london" – the most probable query would be "flew from London" as form is not at all suitable/probable in between flew and London. (Keeping the London lower case is intentional)

e. How Single-Pass In-Memory Indexing (SPIMI) gain efficiency over Blocked sort-based indexing? Explain.

Single Pass In-Memory Indexing (SPIMI) gain efficiency over BSBI by collecting terms directly into a dictionary and taking dynamic posting list. Hence avoiding a sorting need to the global term collection. BSBI works in three steps: collects (term, docID) pairs, sorts them, then writes blocks. On the other hand, SPIMI – directly collect terms in a dictionary structures and with dynamic posting list maintenance. Hence no need to perform a complete term sort.

<The End.>